

T1 - COMPUTAÇÃO PARALELA

Paralelização de uma aplicação com OpenMP

Um path tracer estilo smallpt (Kevin Beason) e renderização Monte Carlo de
uma Cornell Box

Bernardo Klein Heitz

AMD Ryzen 5 3600 (6 núcleos / 12 threads) · gcc 11.4 · Ubuntu 22.04 (WSL2)

1

A APLICAÇÃO

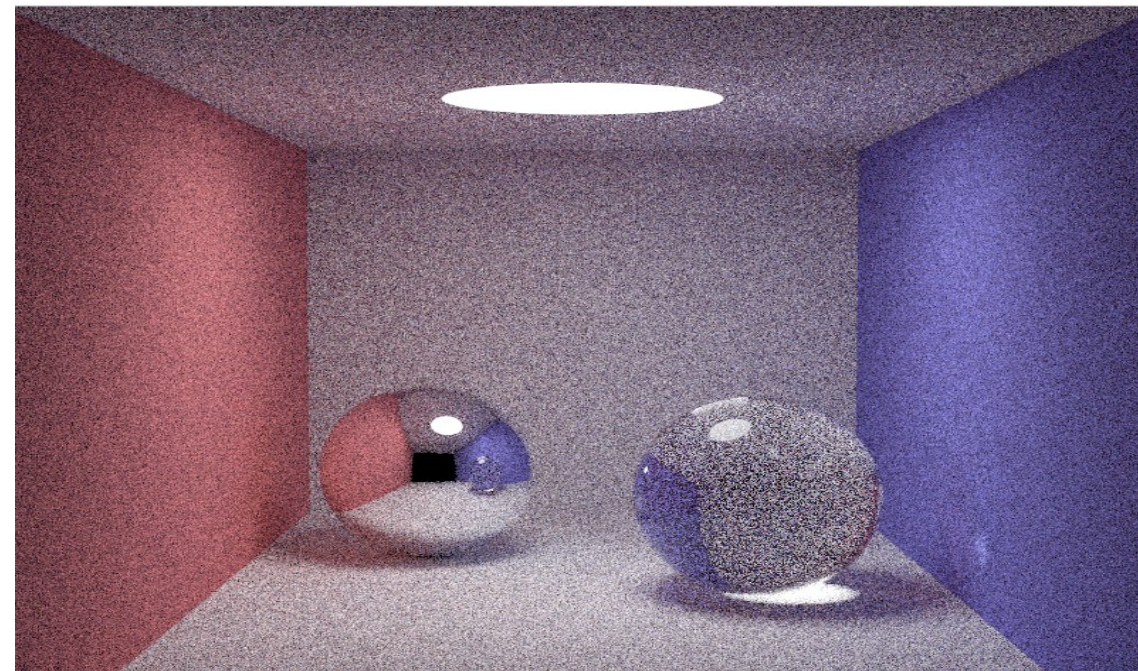
O que é o smallpt

Path tracer mínimo do Kevin Beason (2008), reescrito em C11 (padrão ISO da linguagem C de 2011). Não é um motor de jogos: É um integrador de luz.

Para cada pixel, o programa dispara raios da câmera, deixa ricochetejar na cena e soma a cor da luz que o caminho coletou. Um único caminho é ruidoso; A média de muitos caminhos vira a imagem.

A cena é a Cornell box: Parede vermelha, parede azul, esfera de espelho, esfera de vidro e luz no teto. As “paredes” são esferas gigantes.

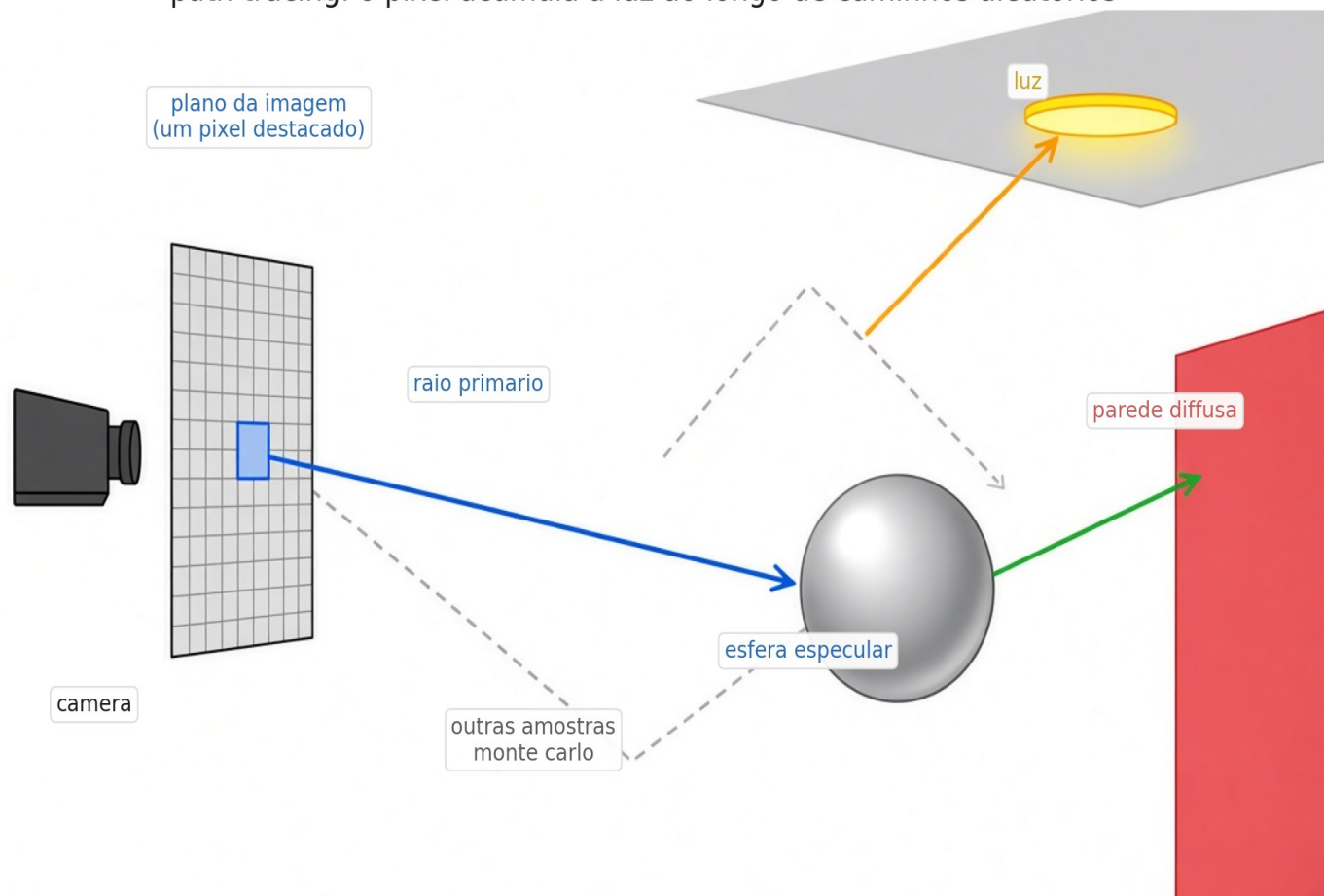
Entrada: -w largura, -h altura, -s spp (amostras por subpixel). Custo proporcional a $w \cdot h \cdot spp$. referência: 800×600 , $spp=32 \rightarrow T1 \approx 54$ s.



por que este programa $T1 \approx 54$ s 1 laço por linha
não trivial (espelho, vidro, monte carlo)

Como um pixel ganha cor

path tracing: o pixel acumula a luz ao longo de caminhos aleatórios



Path tracing:

O raio sai da câmera, atravessa um pixel da imagem e rebate na cena. Em parede difusa o próximo rumo é aleatório; No espelho reflete; No vidro pode refratar. A cor do pixel é a luz acumulada nesse caminho.

Monte carlo:

Estima um integral difícil (a luz que chega no pixel) pela média de amostras aleatórias. Um caminho só é uma amostra ruim; Milhares no mesmo pixel convergem para a imagem certa.

-w -h -s

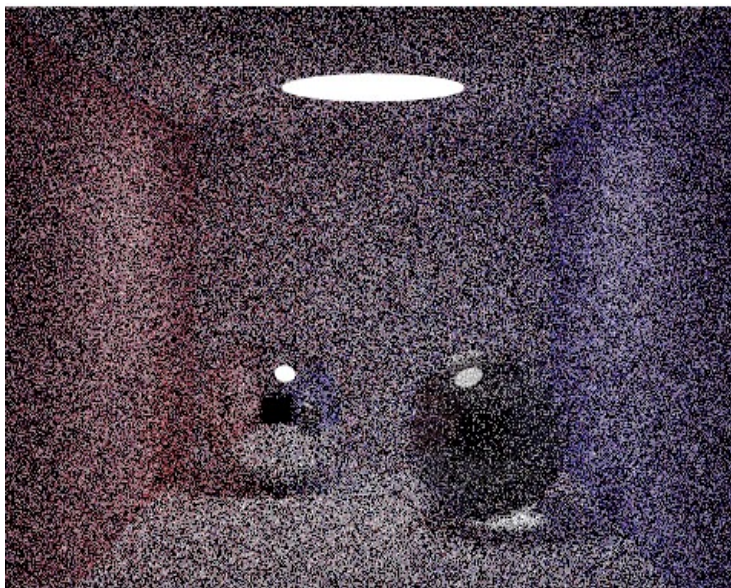
-w = largura em pixels. -h = altura. -s = spp = amostras por subpixel (há 2×2 subpixels, então 4·spp caminhos por pixel). Dobrar o spp dobra o tempo.

Renderização: mais amostras, menos ruído



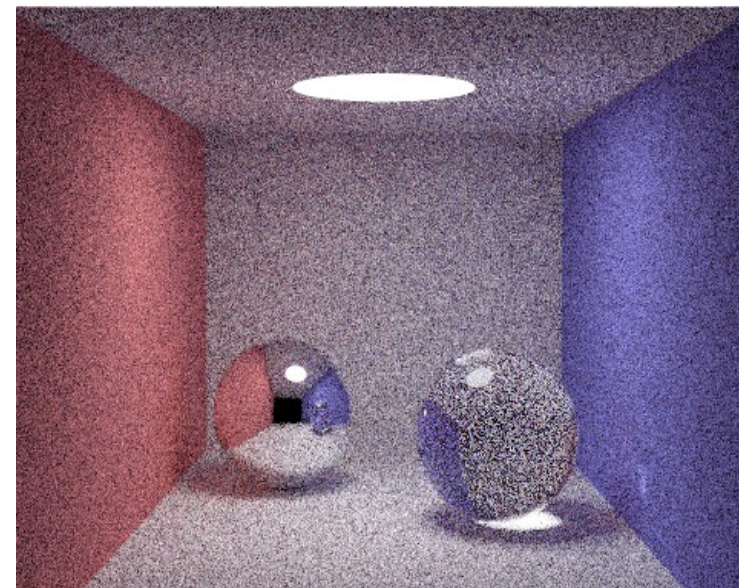
spp = 1

4 caminhos/pixel. Quase só ruído: a média ainda não estabilizou.



spp = 4

16 caminhos/pixel. A sala aparece, mas o granulado continua forte.



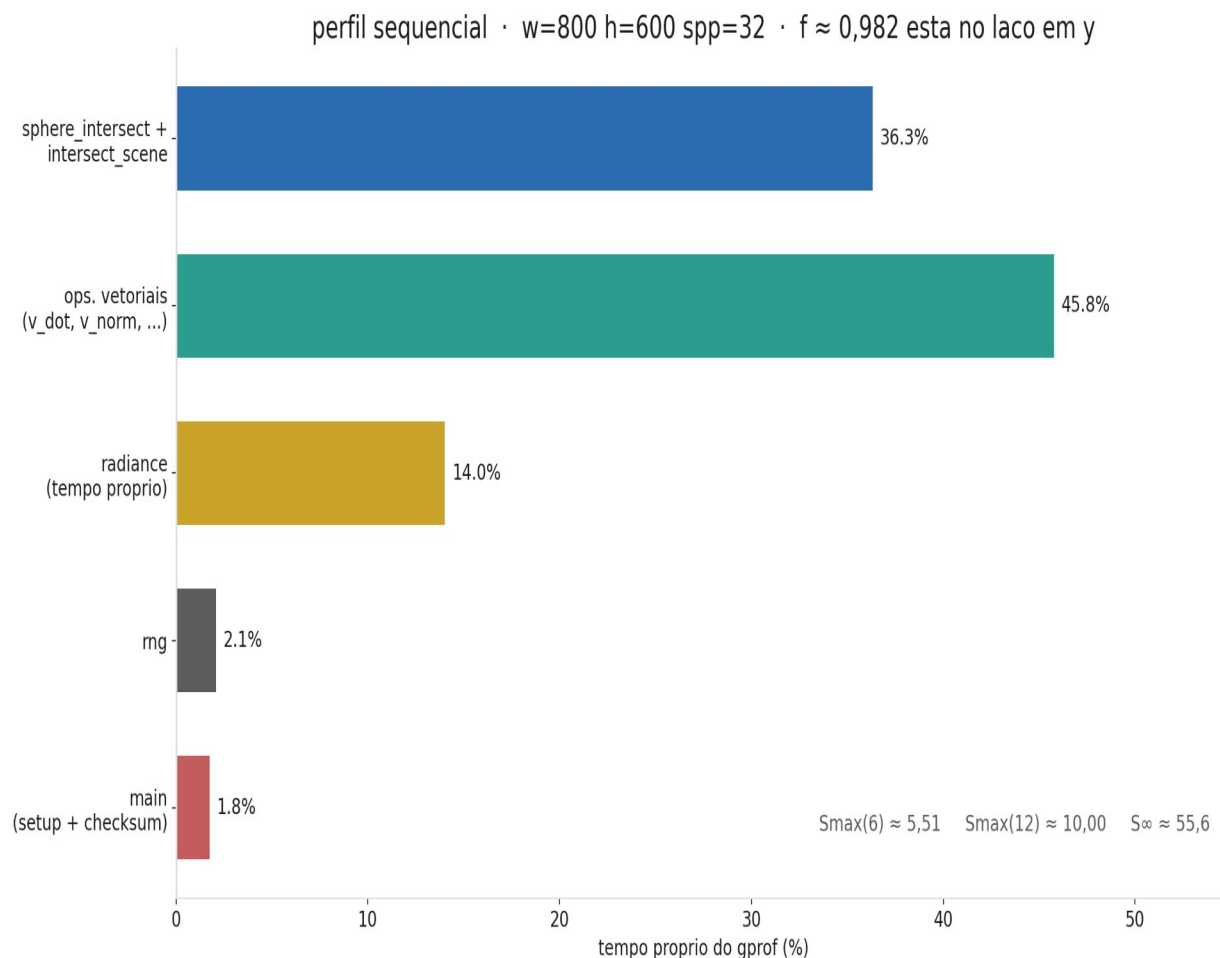
spp = 32 (referência)

128 caminhos/pixel. Mesma entrada do relatório. T1 ≈ 54 s.

4

PERFIL DE EXECUÇÃO

Onde paralelizar: teto de Amdahl



Leitura do gráfico:

Tempo próprio do gprof na versão sequencial. Quase tudo (interseção, vetores, radiance, rng) é chamado de dentro do laço em y. Só o main (~1,8%) fica de fora: Setup e checksum.

O que é o teto de amdahl:

Se uma fração f é paralelizada e 1-f permanece serial, o speed-up com p threads não passa de $S(p) = 1 / ((1-f) + f/p)$. A parte serial não encolhe, por mais threads que se coloque.

Aqui f ≈ 0,982. teto em 6 threads ≈ 5,51; Em 12 ≈ 10; com infinitas threads $S_{\infty} = 1/(1-f) \approx 55,6$. medimos 7,44× em 12 — abaixo de 10. A saturação não é essa fração serial.

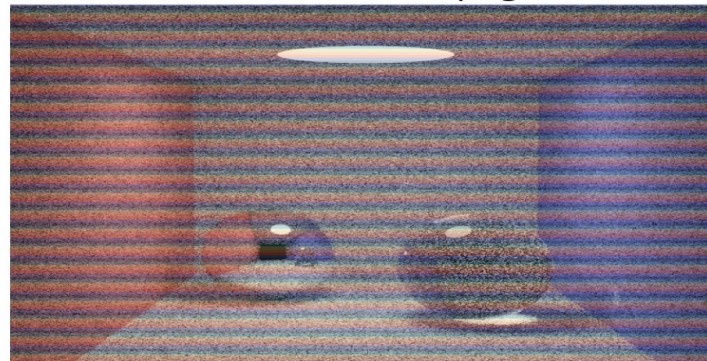
Unidade de trabalho: uma linha da imagem

eixo de paralelismo: cada linha é independente · sem atomic / critical / reduction

static: faixas contíguas · t0 pega y=0 (base da imagem)



dynamic,4: blocos de 4 linhas, pegos sob demanda



thread 0 thread 1 thread 2 thread 3 thread 4 thread 5

static (esquerda):

Parte a imagem em faixas contíguas e fixas. A thread 0 fica com o chão (y=0), a última com o teto. Se uma thread atrasa, no wsl2, isso acontece por preempção e o bloco inteiro dela trava e as outras não pegam o trabalho dela.

dynamic,4 (direita):

As linhas entram num estoque comum em blocos de 4. A thread que termina pega o próximo bloco. O atraso de uma é absorvido pelas outras. Foi a política adotada. Cada linha continua independente: Escrita só em `c[y-w+x]`, rng privado, sem atomic.

As cláusulas e o porquê de cada uma

```
#pragma omp parallel default(none) \
    shared(a, c, cam_o, cam_d, cx, cy,
           stat, threads_used)
{
    #pragma omp for schedule(runtime) nowait
    for (int y = 0; y < a.h; ++y) {
        Rng rng;
        rng.s = a.seed ^ ((y+1)*0x9E37...);
        for (int x = 0; x < a.w; ++x) { ... }
    }
}
```

default(none):

Obriga listar o escopo de cada variável. Se algo deveria ser private e não foi declarado, o compilador recusa, não deixa a condição de corrida escondida.

Shared:

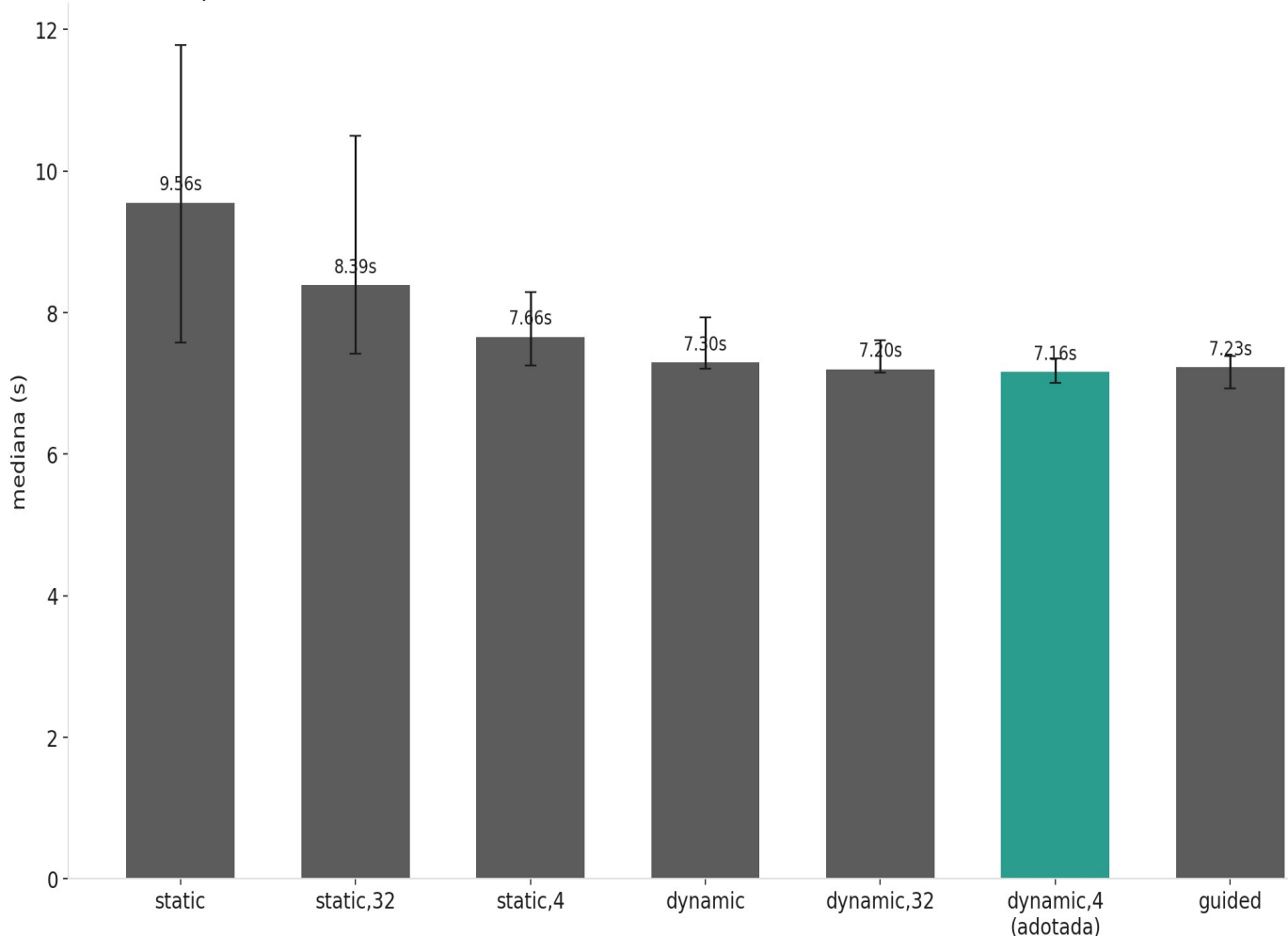
Só entradas (a, câmera), o buffer c (cada thread escreve numa faixa disjunta) e stat[tid] com padding de 64 B contra falso compartilhamento.

schedule(runtime) e nowait:

A política vem de OMP_SCHEDULE, então comparamos static/dynamic/guided sem recompilar. Nowait tira a barreira do for: O tempo por thread é a carga real. Sem reduction: não há acumulador no laço quente; O checksum é depois.

7 políticas testadas, 12 threads e 10 execuções

políticas de escalonamento · 12 threads · 10 execuções · barra = min/max



Leitura do gráfico:

Mediana de 10 execuções em 12 threads. A haste é o intervalo min-max. Barra mais baixa e haste curta = mais rápido e mais estável.

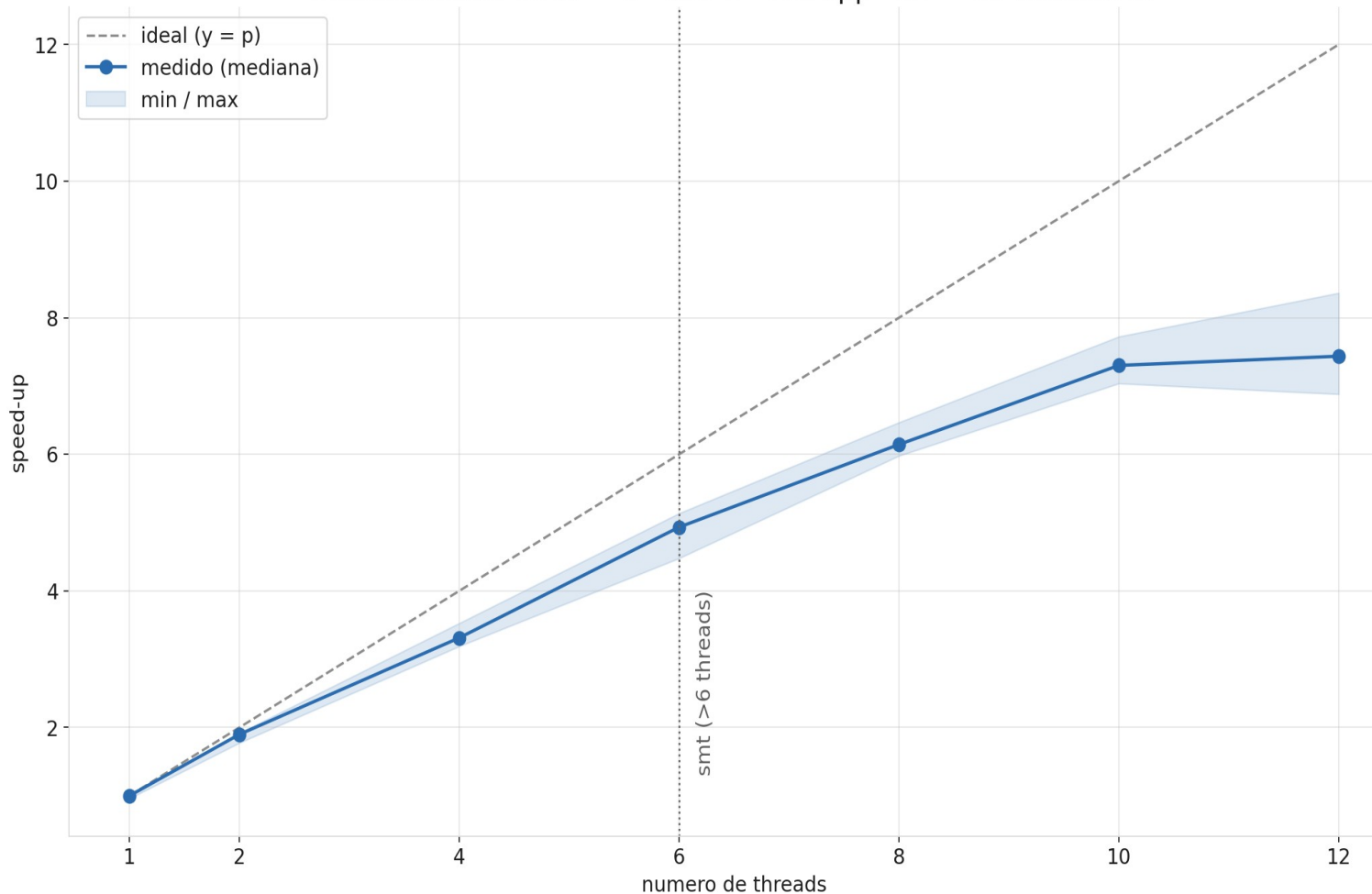
Por que dynamic,4 (verde):

Menor mediana (7,16 s) e menor dispersão (7,01–7,35 s). Static sem chunk é o pior e o mais volátil (7,57–11,78 s): No wsl2 uma thread preemptada trava o bloco fixo. Dynamic,4 redistribui e absorve o atraso.

O algoritmo em si desbalanceia < 1% entre threads. O fator decisivo não é o espelho vs. O fundo: É a robustez à preempção do sistema.

Speed-up: acompanha Amdahl até 6 threads

escalabilidade forte · w=800 h=600 spp=32 · 10 execucoes



Leitura do gráfico:

Eixo x = threads (1, 2, 4, 6, 8, 10, 12). Linha cinza = speed-up ideal ($y = p$). azul = mediana medida. faixa clara = min/max das 10 execuções. Linha pontilhada em 6 = início do smt.

Até 6 núcleos físicos:

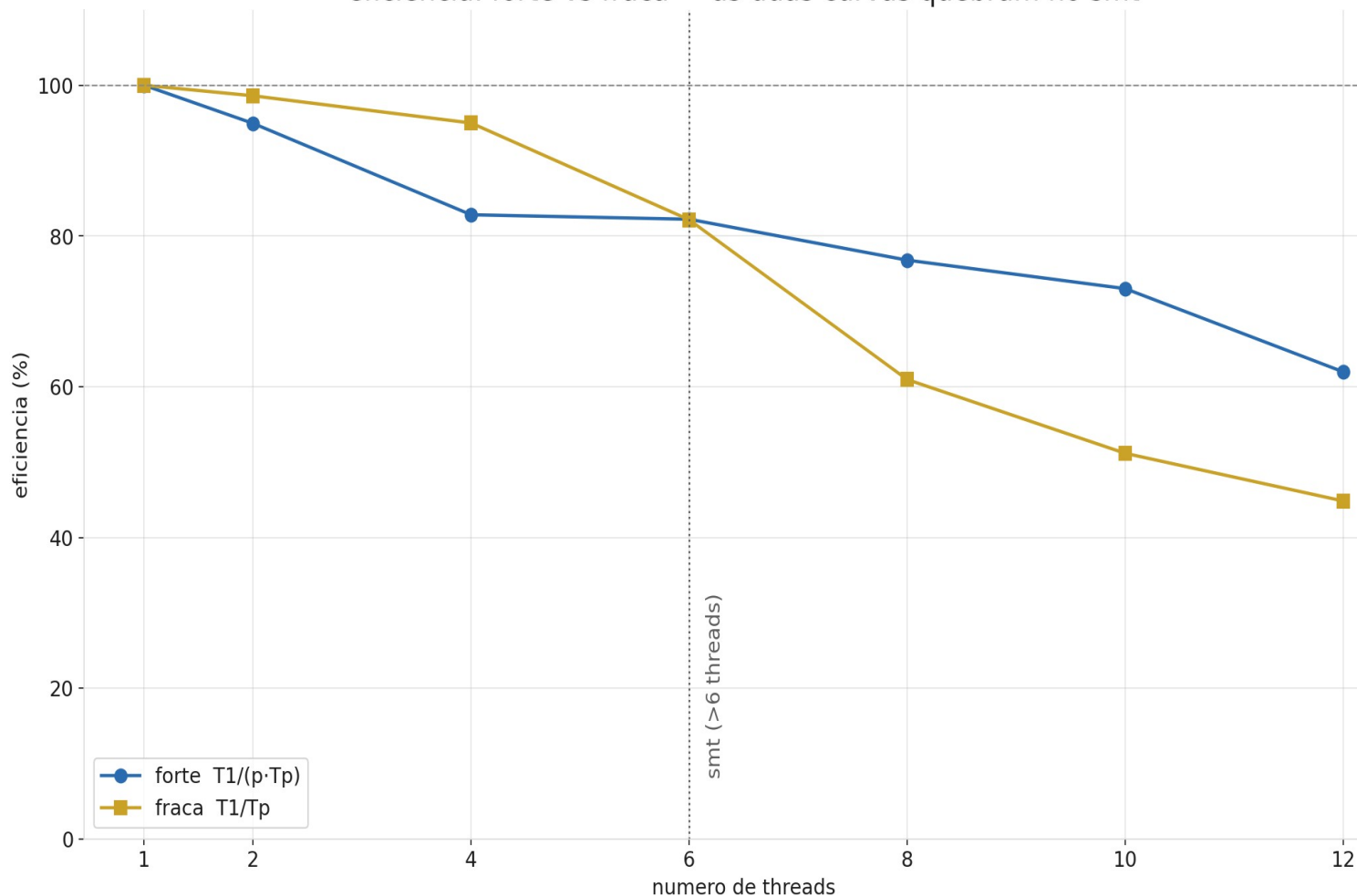
Speed-up 4,93 (teto de amdahl 5,51), eficiência 82,2%. A curva acompanha o ideal de perto.

Depois de 6:

Duas threads no mesmo núcleo disputam ponto flutuante. Em 12: 7,44x e 62%. $7,44 < \text{teto } 10 \rightarrow$ a quebra é smt, não a fração serial.

Eficiência: ambas quebram ao entrar no SMT

eficiencia: forte vs fraca · as duas curvas quebram no smt



Leitura do gráfico:

Azul = eficiência da forte (trabalho fixo).
Amarelo = eficiência da fraca. 100% = o tempo não cresce ao colocar mais threads. A linha pontilhada em 6 marca o smt.

Regra da fraca: $spp = 32 \cdot p$:

W e h ficam 800×600. o spp cresce com as threads porque o custo é linear em spp. O trabalho por thread permanece ~constante.

O que acontece:

Até 4 threads a fraca quase horizontal (55,6 → 58,6 s). Em 12: 124 s (44,8%). Cada núcleo recebe duas cargas completas, então a fraca cai mais que a forte. As duas quebram no mesmo ponto: A máquina, não o algoritmo.

CONCLUSÃO

De 54 s para 7,3 s numa máquina que já temos

7,44×

Speed-up em 12 threads

82,2%

Eficiência em 6 núcleos físicos

Dynamic, 4

Política escolhida por medição

O que os números dizem: de 54 s para 7,3 s (7,44×) numa máquina que já temos. O ponto doce são os 6 núcleos físicos (82,2%). As 12 threads lógicas ainda ajudam, mas com eficiência 62%.

A saturação não é amdahl. $f = 0,982$ permitiria ~10× em 12 threads; medimos 7,44×. A causa é smt (duas threads disputam o ponto flutuante no mesmo núcleo) somada à preempção do wsl2. Por isso adotamos dynamic,4: Redistribui blocos quando uma thread atrasa.

A versão paralela é equivalente à sequencial (checksum rgb idêntico de 1 a 12 threads e nas 7 políticas). Tudo reproduzível: `make all`, `scripts/run_all.sh` e `scripts/plot.py`.